

Explique com suas próprias palavras os seguintes conceitos:

Orientação a Objetos:

Um conceito criado para ajudar na organização em sistemas de programação. Basicamente representa elementos do mundo real como objetos dentro do sistema. Facilitando assim a organização do código.

Classe:

Um modelo ou uma estrutura que define como um objeto (algum elemento do mundo real) será. Dentro dele tem atributos (características) e métodos (comportamentos). É como se fosse uma forma para fazer biscoito ou bolo.

Ex: uma classe ALUNO pode ter NOME, MATRÍCULA, NOTA

Objeto:

É uma instância da classe, é o que nasce a partir da classe. Cada aluno é um objeto diferente.

Ex: ALUNO 1

NOME: Agabo Monteiro

MATRICULA: 113404

NOTA: 10

Atributo:

Atributo é uma variável que pertence a uma classe e representa as características de um objeto. Ou seja é uma informação do objeto.

NOME, MATRICULA, NOTA são atributos do objeto/classe

Método:

É uma função definida de uma classe que representa uma ação ou comportamento do objeto. Basicamente definem o que o objeto pode fazer.

Ex: CALCULAR MÉDIA

MOSTRAR DADOS

ATUALIZAR NOTAS

Construtor:

Método especial utilizado para criar e inicializar objetos de uma classe. Ele é chamado automaticamente quando um objeto é instanciado. Basicamente é o que cria o objeto.

Ex:

```
// Construtor dentro da classe Aluno
```

```
Aluno(String nome, int matricula) {
```

```
this.nome = nome;
```

```
this.matricula = matricula;  
}
```

//criando um objeto usando o construtor
Aluno aluno1 = new Aluno("João", 123);
New = cria o objeto;
Construtor é chamado automaticamente;
Ele coloca os valores dentro do objeto.

Usando uma analogia simples é como preencher a ficha quando o aluno entra na escola.

Modificadores de Acesso:

Os modificadores de acesso controlam quem pode acessar atributos e métodos de uma classe.

Principais modificadores:

public – acessível de qualquer lugar

private – acessível apenas dentro da própria classe

protected – acessível dentro do mesmo pacote ou por classes que herdam dela

default – acessível apenas dentro do mesmo pacote

Eles ajudam a proteger os dados do sistema.

Encapsulamento

Encapsulamento é o princípio de **proteger os dados de uma classe**, permitindo que eles sejam acessados apenas por métodos específicos.

Normalmente os atributos são declarados como **private** e acessados por métodos **get** e **set**.

Isso evita que os dados sejam modificados de forma incorreta.

Como se fosse um cofre e para mexer direto no dinheiro dele precisa usar a chave certa.

Abstração

A abstração consiste em mostrar apenas as informações essenciais de um objeto e esconder detalhes desnecessários.

Ela permite representar elementos complexos do mundo real de forma mais simples dentro do sistema.

Exemplo:

Em um sistema acadêmico, um objeto Disciplina pode ter apenas nome e carga horária, sem representar todos os detalhes administrativos.

Herança

Herança é um mecanismo que permite que uma classe herde atributos e métodos de outra classe.

A classe que herda é chamada de subclasse, e a classe que fornece as características é chamada de superclasse.

Classe **Pessoa**

tem:

- nome
- idade

Classe **Aluno** herda de Pessoa.

Então o aluno já ganha:

- nome
- idade

sem precisar escrever tudo de novo.

Polimorfismo (overload e override)

Polimorfismo significa capacidade de um método assumir diferentes comportamentos.

Overload (Sobrecarga)

Ocorre quando existem métodos com o mesmo nome, mas com parâmetros diferentes dentro da mesma classe.

Exemplo:

calcularMedia(int nota1, int nota2) (2 notas)

calcularMedia(int nota1, int nota2, int nota3) (3 notas)

Override (Sobrescrita)

Ocorre quando uma **classe filha redefine um método da classe pai**, modificando seu comportamento.

Exemplo:

Uma classe **Aluno** pode sobrescrever um método da classe **Usuário**.

Exemplo:

Pessoa → falar()

Aluno → falar() diferente

Classe Abstrata

Uma **classe abstrata** é uma classe que não pode ser instanciada diretamente.

Ela serve como base para outras classes.

Pode possuir:

- métodos completos
- métodos abstratos (sem implementação)

As subclasses devem implementar os métodos abstratos.

Exemplo:

Classe **Animal**

Ela pode dizer que todo animal:

- faz som

Mas cada animal faz um som diferente.

Interface

Uma interface define um conjunto de métodos que uma classe deve implementar.

Ela funciona como um contrato:

qualquer classe que implemente a interface precisa implementar todos os métodos definidos nela.

Interfaces são usadas para garantir que diferentes classes sigam um mesmo padrão de comportamento.

ExInterface **Voador**

Quem usar precisa ter:

- voar()

Então:

- pássaro pode voar
- avião pode voar